

MODULE 9 · TROUBLESHOOTING

Pod Failures

Read the status, then the events

Most incidents start with a Pod that won't run

The Pod's STATUS names the failure class; the **events** and **last state** name the cause. Read them in order and most Pod problems solve themselves.

Five failures cover the vast majority: **CrashLoopBackOff**, **ImagePullBackOff**, **Pending**, **CreateContainerConfigError**, **Init failures**.

BY THE END YOU CAN

- Map a status to its likely cause
- Drive describe → logs
- Diagnose an init-container failure

Ask the questions in order



Each stage has its own failure and its own fix. Don't jump to logs until the container has actually started — earlier failures never reach the logs.

What the status is telling you

STATUS	COMMON CAUSE	GO-TO COMMAND
<code>CrashLoopBackOff</code>	App exits non-zero, bad command, missing dep	<code>logs</code> + <code>describe</code> (Last State)
<code>ImagePullBackOff</code>	Wrong image name/tag, registry auth	<code>describe</code> (events)
<code>Pending</code>	No node fits — requests too big, taints	<code>describe</code> (FailedScheduling)
<code>CreateContainerConfigError</code>	Missing ConfigMap / Secret	<code>describe</code> (events)
<code>Init:Error</code>	Init container exits non-zero	<code>logs -c <init></code>

DIAGNOSE

Two commands, in this order

- **describe first** — Events + Last State cover scheduling, pull, and exit-code failures
- **logs second** — once the container has started, to see what the app said

ORDER MATTERS

Pending and image-pull failures never produce logs — describe is the only window.

```
$ kubectl describe pod NAME
# Events: FailedScheduling / pull
# Last State: Terminated, exit code

$ kubectl logs NAME
$ kubectl logs NAME -c INIT
```

The exit code is the story

The container starts, exits with an error, and the kubelet restarts it with growing back-off. describe shows the **Last State** exit code; logs shows what it printed before dying.

Fix = correct the command/config so the process stays up, then delete & re-apply for an immediate retry.

```
Last State:  Terminated
Reason:      Error
Exit Code:  1
Events:
  Back-off restarting failed
  container
```

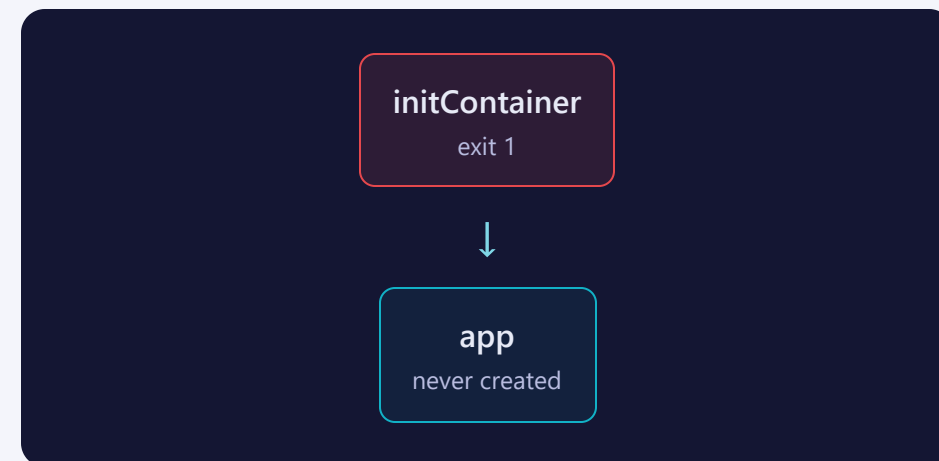
If an init container fails, the app never starts

Status shows `Init:Error / Init:CrashLoopBackOff` and `READY 0/1`.

The main container is still `PodInitializing` — it was never created.

Don't debug the app. The answer is in the **init container's logs**:

```
$ kubectl logs NAME -c prepare-data
```



Where people trip

- **Reading get alone.** "Pending" isn't a cause — describe has the reason.
- **Chasing app logs** on an image-pull or scheduling failure — there are none yet.
- **Ignoring restart back-off.** A fixed manifest still waits for the timer — delete & re-apply.
- **Misreading Init: status** as an app crash.

Commands to keep at hand

```
$ kubectl get pod NAME           # the status class
$ kubectl describe pod NAME      # events + last state = the cause
$ kubectl logs NAME [-c CONTAINER] # what the process said
$ kubectl logs NAME --previous   # the crashed instance
$ kubectl get events --sort-by=.lastTimestamp
```

Tip: status → describe → logs. Nine times in ten the cause is in the events, not the logs.

RECAP

Pod triage in one breath

- Status = the failure class; events/last-state = the cause
- Scheduled? → pulled? → started? → running
- describe before logs
- Init failures hide in logs `-c <init>`

HANDS-ON

Now run Lab 9.1
[labs/9.1/](#)

Next: [9.2 — Image Pull Problems](#)